

Nikol Vidaković RA232/2023
Nedeljko Ćuso RA175/2023
Valerija Teovanović RA187/2023
Vladimir Trkulja RA233/2023

Predmet: Upravljački algoritmi u realnom vremenu
Novi Sad, jul 2026.

Sistem za upravljanje liftom SIL/HIL simulacija

Dokumentacija

Sadržaj

Predgovor	2
Uvod	4
Uputstvo za korišćenje front-panela	5
Model dinamike lifta	6
Regularizacija pozicije	8
Automat stanja	9
Raspodela na dva uređaja (HIL)	12
Korisnički panel	13
Završetak programa.....	14
Zaključak.....	15

Predgovor

Upravljanje liftom predstavlja klasičan primer sistema u kom se prepliću zahtevi za bezbednošću, predvidivim ponašanjem u realnom vremenu i jednostavnošću korisničkog iskustva. Iako je reč o naizgled jednostavnom uređaju, njegova ispravna realizacija zahteva pažljivo projektovan automat stanja, adekvatan regulator pozicije i, pre svega, metodologiju testiranja koja garantuje da će sistem raditi ispravno i onda kada softver napusti bezbedno okruženje personalnog računara i pređe na stvaran, mrežno raspoređen hardver.

Upravo iz tog razloga, razvoj ovog projekta sproveden je kroz dve jasno odvojene faze. U prvoj fazi (SIL—Software in the Loop) čitav sistem, i model fizike lifta i upravljačka logika, izvršava se zajedno, softverski, na jednom računaru — ovo omogućava brzo otkrivanje i ispravljanje grešaka u logici bez rizika po hardver. Tek kada je ponašanje u ovoj fazi u potpunosti potvrđeno, sistem se raspoređuje na dva fizički odvojena NI kontrolera koji komuniciraju isključivo preko mreže (HIL—Hardware in the Loop), čime se verno simulira scenario u kom su upravljačka jedinica i sam uređaj fizički razdvojeni, baš kao što bi bio slučaj kod pravog lifta i njegovog kontrolnog ormana.

Ovaj rad opisuje projektovanje i realizaciju ovakvog sistema u razvojnom okruženju LabVIEW. Poseban akcenat stavljen je na analitičko izvođenje parametara regulatora, na robusnost automata stanja u prisustvu grešaka i komande za zaustavljanje, kao i na praktične korake neophodne za uspešnu raspodelu aplikacije na dva odvojena kontrolera.

Uvod

Osnovu realizacije ovog projektnog zadatka čini simulacija fizičkog ponašanja lifta i njegovog upravljanja, raspoređena na dva sbRIO/cRIO kontrolera koji međusobno komuniciraju preko mreže. Za razliku od projekata koji se oslanjaju na akviziciju stvarnog analognog signala, ovde je fokus na raspoređenim upravljačkim sistemima u realnom vremenu — na tome kako se logika koja u simulaciji radi besprekorno na jednom računar ponaša kada se razdeli na dva nezavisna, mrežno povezana uređaja, od kojih svaki radi pod sopstvenim real-time operativnim sistemom.

Fizika lifta namerno je modelovana pojednostavljeno, sistemom prvog reda — realizacija verne, detaljne mehaničke simulacije (masa kabine, karakteristika motora, trenje) nije bila cilj zadatka. Umesto toga, pažnja je posvećena tome da se odziv sistema analitički i kvalitetno reguliše, kao i da se cela arhitektura — automat stanja, regulator, komunikacija preko deljenih promenljivih — ispravno prenese sa jedne mašine na dve.

Glavni cilj projekta bio je razvoj kompletne aplikacije za upravljanje liftom u LabVIEW okruženju, kroz tri odvojena programa: model dinamike lifta, upravljačku aplikaciju (automat stanja sa regulatorom) i korisnički panel koji predstavlja poziv i unutrašnju tablu lifta. Program obuhvata module za regulaciju pozicije metodom postavljanja polova, robustan automat sa četiri stanja i bezbednosnim mehanizmom trenutnog gašenja, kao i korisnički interfejs koji u realnom vremenu prikazuje poziciju, kretanje i stanje sistema.

Uputstvo za korišćenje front-panela

Pre početka rada, potrebno je pokrenuti sva tri VI-ja tačno sledećim redosledom:

Model_Lifta.vi → Automat_Lift_sbRIO.vi → PC_HMI_Lift.vi

Ovaj redosled nije proizvoljan — svaki VI, pri pokretanju, eksplicitno upisuje podrazumevane vrednosti mrežnih promenljivih koje čita, te je važno da model i automat budu aktivni pre nego što korisnik počne da šalje komande preko HMI-ja.

Korisnički panel podeljen je na dva glavna dela:

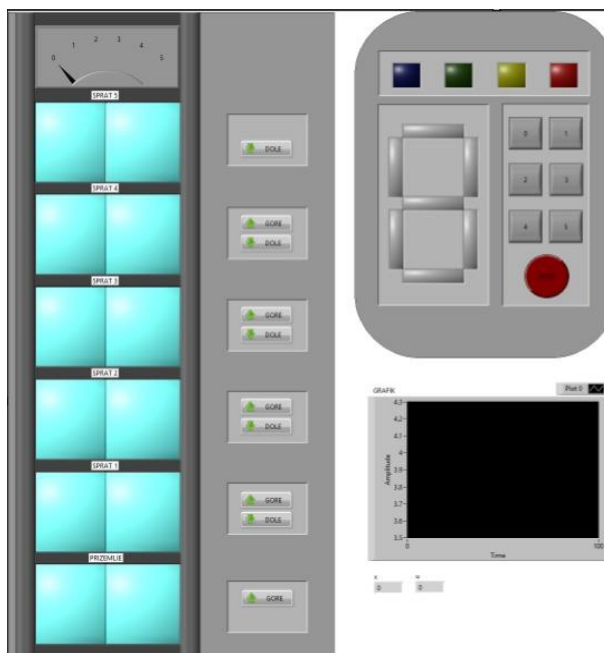
- komandnu tablu (unutrašnji pozivi i indikatori stanja)
- prikaz zgrade (spoljašnji pozivi i vizuelizacija položaja lifta).

Komandna tabla sadrži četiri LED indikatora stanja automata: Idle, Moving, DoorOpen i Shutdown — od kojih je u svakom trenutku upaljen tačno jedan, odgovarajući aktivnom stanju automata. Ispod njih nalazi se digitalni displej koji prikazuje trenutno izabrani ciljani sprat. Šest dugmadi (0–5) služi za izbor destinacije iz same kabine, dok STOP dugme trenutno prekida rad sistema i prebacuje automat u stanje Shutdown. Dugmad za izbor sprata iznutra namerno su ograničena: aktivna su tek nakon što je lift pozvan spolja, i to samo za one spratove koji odgovaraju smeru tog poziva.

Prikaz lifta u zgradi predstavljen je kao niz "prozora", u vidu dva pravougaona polja koja menjaju boju kada se lift nalazi na tom spratu, čime se vizuelno prati kretanje kabine kroz zgradu. Korišćen je i Meter indikator sa kazaljkom koji prikazuje tačan sprat lifta. Desno od prikaza vrata lifta nalaze se spoljašnji pozivni tasteri — GORE i DOLE za svaki sprat, pri čemu krajnji spratovi (najniži i najviši) poseduju samo po jedan mogući smer poziva.

Posebno, van glavnog prikaza zgrade, nalazi se i grafikon, koji u realnom vremenu iscrtava vrednosti pozicije (x) i upravljačkog signala (u) kroz vreme (X-osa: Time, Y-osa: Amplitude), omogućavajući uvid u dinamiku odziva sistema tokom kretanja lifta.

Pri gašenju panela, sve LED diode se eksplicitno vraćaju u ugašeno stanje.



Slika 1: Front panel PC_HMI_Lift.vi dugmad za pozive, grafik pozicije i LED indikatori stanja

Model dinamike lifta

Umesto detaljnog fizičkog modela, pozicija lifta opisana je jednom jednačinom prvog reda:

$$T \cdot \left(\frac{dx}{dt} \right) + x = K \cdot u$$

gde je x pozicija lifta (izražena u jedinicama sprata), a u upravljački signal koji dolazi iz regulatora u Automatu i predstavlja "komandu" modelu koliko snažno i u kom smeru da menja poziciju.

Nakon diskretizacije jednačina implementirana u Formula Node-u glasi:

$$x[k + 1] = x[k] + \left(\frac{T_s}{T} \right) \cdot (K \cdot u[k] - x[k])$$

U Formula Node-u ova jednačina je implementirana sa već uvrštenim vrednostima parametara ($T_s = 0.1$ s, $T = 5$, $K = 0.55$, pa je $T_s/T = 0.02$):

$$x_{new} = x_{prev} + 0.02 \cdot (0.55 \cdot u - x_{prev})$$

Konstanta K nije bila deo prvobitne formule — dodata je naknadno kako bi se ublažio nagli, gotovo trenutni skok upravljačkog signala u trenutku kada je lift daleko od cilja, s obzirom na to da je na samom početku putovanja greška najveća, pa regulator inicijalno zahteva veliku "silu".

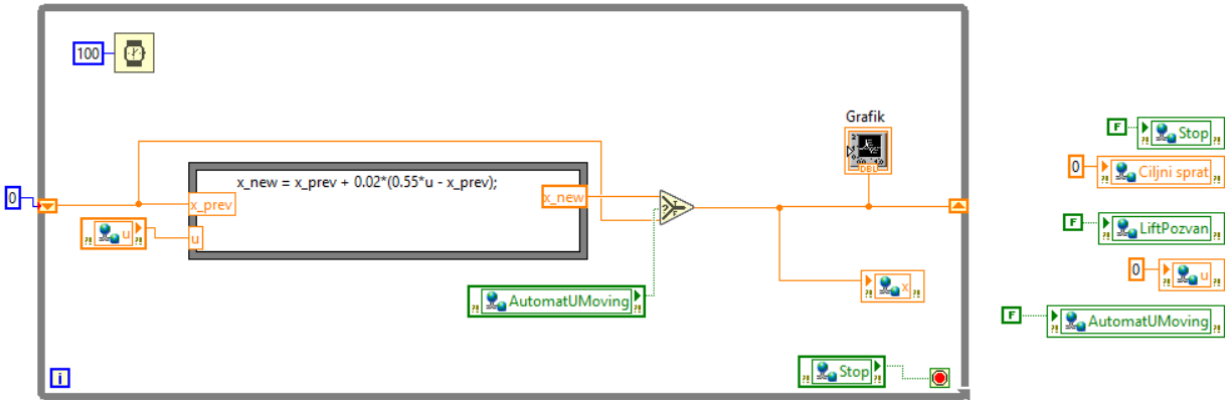
Formula Node se nalazi unutar While petlje sa periodom čekanja od 100 ms (Wait funkcija), a ulazne vrednosti x_{prev} i u čita preko Read Variable funkcija sa mreže, dok izlaznu vrednost x_{new} prosleđuje dalje u istoj iteraciji.

Poseban problem koji se javio tokom razvoja bilo je "curenje" pozicije lifta unazad ka nuli, čak i kada automat miruje — posledica same formule, koja pri $u = 0$ i dalje polako vuče x ka nuli. Rešeno je uvođenjem deljene promenljive AutomatUMoving, kojom automat modelu javlja da li je trenutno u stanju Moving. Ova provera realizovana je preko Select funkcije: ako je AutomatUMoving = True, na izlaz prosleđuje se novoizračunata vrednost x_{new} ; u suprotnom se prosleđuje x_{prev} , čime se prethodna pozicija zadržava nepromenjenom sve dok automat ponovo ne pređe u stanje Moving.

Vrednost x se, nezavisno od ishoda Select funkcije, upisuje i na Grafik indikator, čime se omogućava kontinuirano praćenje pozicije kroz vreme, uporedo sa upravljačkim signalom u .

Uslovni terminal While petlje povezan je na promenljivu Stop — kada je Stop = True, petlja se zaustavlja, o čemu dodatno signalizira i indikator na bloku dijagramu.

Pre ulaska u While petlju, VI eksplicitno upisuje podrazumevane vrednosti svih deljenih promenljivih koje čita ili na koje bi mogao uticati zaostala vrednost sa Shared Variable Engine-a: Stop, Ciljnisprat, LiftPozvan, u , AutomatUMoving. Ovim se sprečava scenario u kom bi model, pri ponovnom pokretanju, "nasledio" vrednosti iz prethodne sesije (npr. Stop zaglavljen na True), pre nego što Automat i PC HMI uopšte stignu da postavе svoje vrednosti.



Slika 2: Block diagram Model_Lifta.vi — Formula Node, shift register pozicije i shared variable čvorovi u, x, AutomatUMoving, Stop

Regularizacija pozicije

Upravljački signal računa se preko PI zakona:

$$\begin{aligned}
 u &= Kp \cdot e + Ki \cdot I \\
 I[k + 1] &= I[k] + Ts \cdot e[k] \\
 e &= CiljniSprat - x
 \end{aligned}$$

D član nije uveden — pokazalo se da za sistem ovog reda PI regulacija sasvim dovoljno eliminiše grešku, bez oscilatornog ponašanja koje bi opravdalo dodatnu komplikaciju.

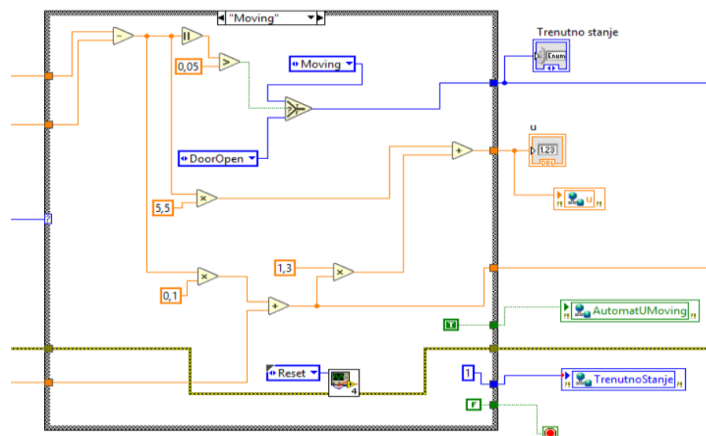
Pojačanja Kp i Ki nisu birana proizvoljno, već izvedena iz zahtevanih polova zatvorene petlje:

$$\begin{aligned}
 Kp &= \frac{2 - a - (p1 + p2)}{a \cdot K} \\
 Ki &= \frac{p1 \cdot p2 - (1 - a - a \cdot K \cdot Kp)}{a \cdot K \cdot Ts} \\
 a &= \frac{Ts}{T}
 \end{aligned}$$

Isprobane su kombinacije polova pre finalnog izbora — dupli, bliski polovi davali su primetno preskakanje cilja pri većim udaljenostima, dok su razmaknutiji, realni polovi dali stabilniji odziv. Usvojene vrednosti: $K_p = 5.5$, $K_i = 1.3$.

Granice za promenu stanja nisu iste u oba smera (namerna histereza):

- Idle → Moving : kada je $|e| > 0.8$, pošto su spratovi celi brojevi i nema potrebe reagovati na sitnija odstupanja
- Moving → DoorOpen : kada je $|e| < 0.05$, jer se ovde traži preciznost — ova granica određuje da li je lift "stvarno stigao"



Slika 3: Deo block diagrama Automat_Lift_sbRIO.vi gde se računa PI regulator — greška e , integralna suma I i izlaz u

Automat stanja

Korišćena je jednopetljna arhitektura automata — Lobby, Guard Clause, glavna Case struktura, Alley — po uzoru na vežbe sa cRIO/sbRIO kontrolerima, gde se komunikacija između slojeva rešava preko deljenih promenljivih, a ne preko queue mehanizma, s obzirom na to da Automat i Model rade na dva fizički odvojena uređaja.

Automat ima četiri stanja:

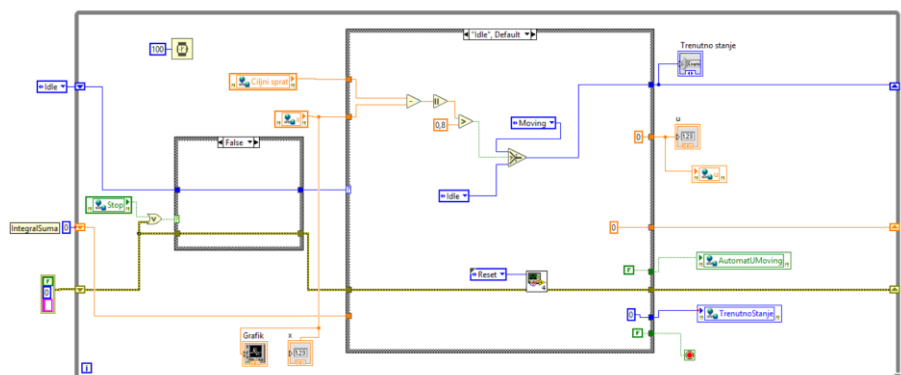
- Idle (čekanje na novi poziv)
- Moving (kretanje ka cilju)
- DoorOpen (lift stigao, vrata otvorena)
- Shutdown (sistem zaustavljen).

Stanje se čuva kao Enum vrednost (Trenutno stanje), a paralelno se upisuje i na mrežnu promenljivu TrenutnoStanje kao U8 broj (0=Idle, 1=Moving, 2=DoorOpen, 3=Shutdown), koju čita PC HMI radi paljenja odgovarajućeg LED indikatora.

Guard Clause je mala Case struktura čiji selektor prati uslov (Stop ILI greška); kada je uslov ispunjen, sledeće stanje se prisilno postavlja na Shutdown, bez obzira šta bi glavna Case struktura inače odlučila.

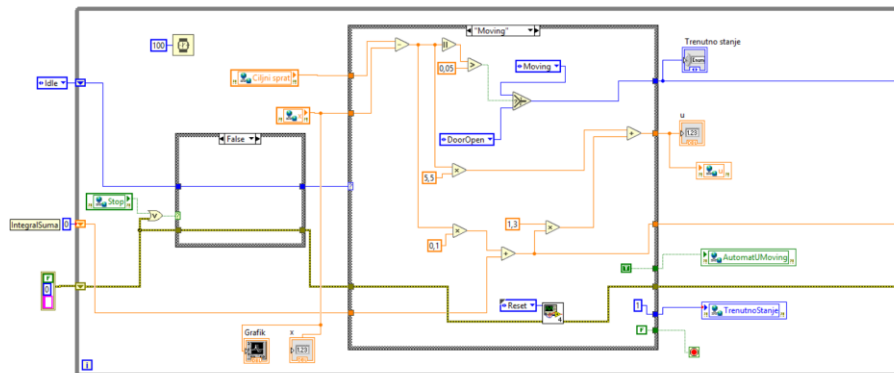
Kroz petlju automata provlače se: trenutno stanje, pozicija x , i integralna suma regulatora (IntegralSuma, shift registar). U HIL verziji, x se više ne pamti lokalno kroz shift registar — automat ga svaki put učitava sa mreže (promenljiva x), jer tu vrednost sada održava model na drugom uređaju.

Idle case: greška e se računa kao Ciljni sprat – x , uzima joj se apsolutna vrednost, i poredi sa 0.8; ako je $|e| > 0.8$, preko Select funkcije se sledeće stanje postavlja na Moving, inače ostaje Idle. Integralna suma se eksplicitno resetuje na 0, bez ovoga bi svako novo putovanje "nasledilo" akumuliranu grešku iz prethodnog, što bi pokvarilo ponašanje regulatora pri sledećem pokretanju. Upravljački signal u ovom stanju iznosi 0, AutomatUMoving = False, a Timer se poziva sa metodom Reset (priprema tajmera za sledeći ulazak u DoorOpen).



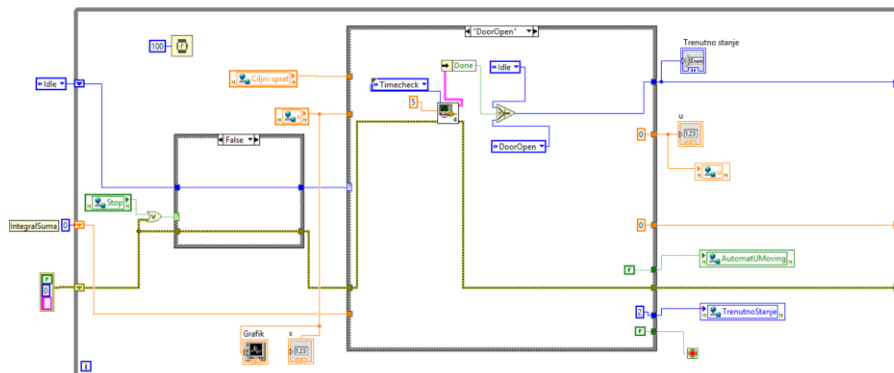
Slika 4: Idle stanje

Moving case: PI regulator aktivno računa $u = K_p \cdot e + K_i \cdot I$ (detaljno opisano u poglavlju "Regulator pozicije"); kada $|e| < 0.05$, preko Select funkcije sledeće stanje postaje DoorOpen, inače ostaje Moving. AutomatUMoving = True, ovo je jedino stanje u kome model zaista ažurira poziciju x . Timer se i ovde poziva sa Reset metodom.



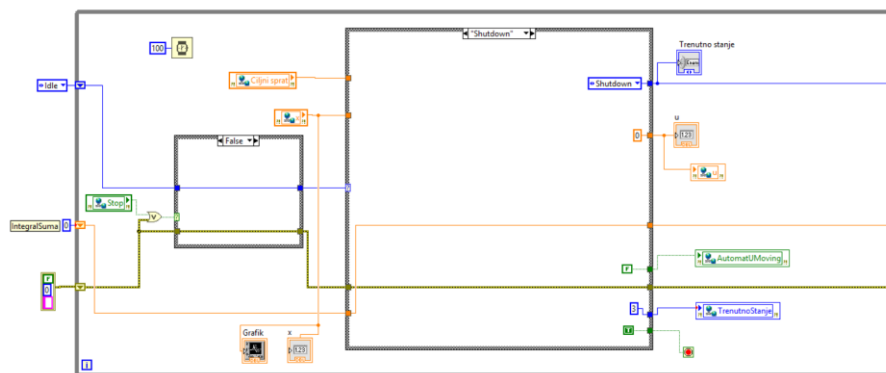
Slika 5: Moving stanje

DoorOpen case: korišćen je namenski Action Engine tajmer umesto standardnog Elapsed Time Express VI-ja — potonji se pokazao nepouzdanim jer se ne resetuje ispravno pri ponovljenim ulascima u isto stanje. Poziva se metoda Timecheck sa Time Target = 5 (sekundi); kada tajmer javi Done, preko Select funkcije sledeće stanje postaje Idle, inače ostaje DoorOpen. Upravljački signal $u=0$, AutomatUMoving = False.



Slika 6: DoorOpen stanje

Shutdown case: sledeće stanje je uvek Shutdown (nema uslovnog grananja), $u=0$, AutomatUMoving = False. Ovo je jedino stanje u kome se signal na uslovnom terminalu While petlje (crveni "stop" indikator) postavlja na True, čime se petlja automata zaustavlja.



Slika 7: Shutdown stanje

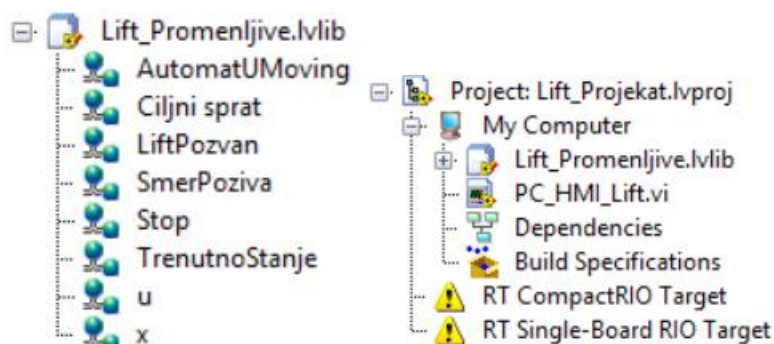
Raspodela na dva uređaja (HIL)

Sistem čine tri odvojena VI-ja, raspoređena na tri različite izvršne jedinice:

- **Model lifta** (Model_Lifta.vi) — simulacija fizike lifta, izvršava se na sbRIO kontroleru;
- **Upravljačka aplikacija** (Automat_Lift_sbRIO.vi) — automat stanja i PI regulator, izvršava se na cRIO kontroleru (naziv fajla je zaostatak iz ranije faze razvoja);
- **Korisnički panel** (PC_HMI_Lift.vi) — korisnički interfejs, izvršava se na personalnom računaru.

FPGA sloj nijednog od dva kontrolera se ne koristi — nema fizičkih senzora ni aktuatora u ovoj verziji projekta, pa je cela razmena podataka softverska, preko mrežnih deljenih promenljivih.

Ovakva podela direktno odražava filozofiju HIL testiranja: upravljačka logika i model fizike ne dele više isti proces niti istu memoriju, već komuniciraju isključivo preko mreže, baš kao što bi to bio slučaj kod stvarnog lifta, gde bi kontrolna jedinica i senzori/aktuatori bili povezani industrijskom mrežom, a ne izvršavani unutar jednog programa.



Slika 8: Project explorer i biblioteke

Naziv	Tip	Upisuje	Čita
u	Double	Automat	Model
x	Double	Model	Automat, Panel
Stop	Boolean	Automat, Panel	Model, Automat
CiljaniSprat	Double	Panel	Automat
AutomatUMoving	Boolean	Automat	Model
TrenutnoStanje	U8 (0-3)	Automat	Panel
LiftPozvan	Boolean	Panel	Panel
SmerPoziva	Boolean	Panel	Panel

Tabela 1: Deljene promenljive

Pošto deljena promenljiva na serveru zadržava poslednju vrednost nezavisno od toga da li je neki VI restartovan, svaki VI koji čita Stop/CiljniSprat pri startu eksplicitno upisuje podrazumevanu vrednost — bez ovoga bi se moglo desiti da sistem "zapamti" da je Stop ranije bio pritisnut, i ostane trajno zaključan u Shutdown-u pri sledećem pokretanju.

Praktično, promenljive se koriste tako što se biblioteka Lift_Promenljive.lvlib otvori, željena promenljiva direktno prevuče iz biblioteke na blok dijagram, nakon čega se čvor podesi u odgovarajući režim (Read ili Write), u zavisnosti od toga da li taj deo aplikacije treba da čita ili upisuje vrednost.

Fizičko povezivanje ostvareno je preko mrežnog kabla između cRIO i sbRIO uređaja; nakon potvrde vidljivosti oba uređaja u NI MAX-u, oba su dodata u stablo LabVIEW projekta, sa odgovarajućim VI-jevima raspoređenim ispod svakog targeta

Korisnički panel

Struktura panela oslanja se na dve nezavisne petlje: jedna, sa Event strukturom, reaguje isključivo na klikove i upisuje odgovarajuće komande na mrežu; druga, bez Event strukture, neprekidno čita trenutnu poziciju i stanje i osvežava prikaz. Ovakvo razdvajanje obezbeđuje da prikaz pozicije i stanja ostane gladak i ažuran u realnom vremenu, nezavisno od toga da li korisnik u tom trenutku klika na neko dugme.

Petlja za osvežavanje prikaza (period čekanja 100 ms) čita promenljivu `TrenutnoStanje` i preko niza `Equal?` poređenja (sa vrednostima 0, 1, 2, 3) pali odgovarajući od četiri LED indikatora — IDLE, MOVING, DOOR OPEN, SHUTDOWN — pri čemu je enkodiranje usklađeno sa automatom (0=Idle, 1=Moving, 2=DoorOpen, 3=Shutdown). Ista petlja čita promenljive `x` i `u` i prosleđuje ih na GRAFIK (waveform prikaz kroz vreme) i na Meter indikator (direktna, nezaokružena pozicija).

Vizuelno otvaranje i zatvaranje vrata (stanje `DoorOpen`) realizovano je promenom `Width` i `Left` property node-ova Boolean elemenata koji predstavljaju "vrata" na svakom spratu — kada je uslov `DoorOpen` (u kombinaciji sa proverom da se lift nalazi na tom spratu) ispunjen, elementi vrata se animiraju kroz promenu širine i pozicije, simulirajući njihovo razmicanje, umesto da se, kako je prvobitno bilo planirano, koristi samo treća boja LED indikatora.

STOP dugme realizovano je kroz zaseban Event Case (Value Change) unutar Event petlje — pri pritisku, upisuje `Stop = True` na mrežu.

Disable logika za Sprat dugmad potvrđuje ranije opisano ponašanje: za svaki sprat (0–5) računa se razlika u odnosu na `CiljniSprat`, poredi se sa nulom (Greater Than), rezultat se AND-uje sa `SmerPoziva` i `LiftPozvan`, invertuje (NOT), i preko `Select` funkcije konvertuje u U8 vrednost (0=Enabled, 1=Disabled) koja se prosleđuje na Property Node "Disabled" odgovarajućeg SPRAT dugmeta. Logika se identično ponavlja za svih šest dugmadi, uz promenu jedino referentne vrednosti sprata.

Završetak programa

Sistem se bezbedno zaustavlja putem Guard Clause mehanizma opisanog u poglavlju "Automat stanja" — bilo pritiskom na Stop, bilo usled greške, automat prelazi u stanje Shutdown, gde se izvršavanje petlje prekida na kontrolisan način,

bez naglog gašenja koje bi moglo ostaviti deljene promenljive u nekonzistentnom stanju.

Zaključak

Projekat je pokazao ceo put od matematičkog modela, preko analitički podešenog regulatora, do funkcionalnog automata i njegove raspodele na realan, mrežno povezan hardver. Nekoliko problema uočenih tokom razvoja — asimetrija usled izostavljene apsolutne vrednosti, nepouzdan standardni tajmer, promenljive koje "pamte" staro stanje — predstavljaju tipične zamke pri prelasku sa jednostavne, jednopoljne simulacije na raspoređeni sistem, i njihovo otklanjanje bio je značajan deo praktičnog rada na projektu.